

QA

AUTOMATION TESTING PROJECT DOCUMENTATION

Goibibo · Holidays Module · End-to-End Web Automation

PROJECT DETAILS

Prepared By	Vanaparathi Deekshith
Website	Goibibo.com — Holidays Module
Automation Tool	Selenium WebDriver (Python)
Frameworks	Pytest · Page Object Model Behave · BDD
Reporting	Allure Reports · HTML · JSON · JUnit
Browser	Google Chrome
Date	24 May 2026

Table of Contents

S NO.	DESCRIPTION	PAGE NO.
1	Executive Overview	1
2	Project Description	2
3	Project Objectives	3
4	Project Workflow	3
5	Test Strategy	5
6	Framework Architecture	
	6.1 : POM Framework	5
	6.2 : BDD Framework	6
7	Tools & Technology Used	7
8	Test Environment	7
9	Test Execution Commands	
	9.1 : Pytest Commands – POM Framework	8
	9.2 : Behave Commands – BDD Framework	8
	9.3 : Allure Report Commands	9
10	Test Output & Evidence	9
11	Allure Test Reports	11
12	Positive Test Cases	13
13	Negative Test Cases	14
14	Challenges Faced During Automation	14
15	Solutions Implemented	15
16	Key Features of the Framework	16
17	Important Framework Files	16
18	Learning Outcomes	17
19	Conclusion	17

List of Figures

FIGURE NO.	TITLE	PAGE NO.
1	Combined BDD & POM workflow	4
2	Goibibo home page	10
3	Holidays search page	10
4	Payment Boundary page	11
5	Allure overview report	12
6	Allure suites view	13

01 Executive Overview

This documentation presents the Goibibo Holidays automation testing project developed using Selenium WebDriver with Python. The project contains two parallel implementations: a code-driven Pytest Page Object Model (POM) framework and a business-readable Behave BDD framework. Both validate the Goibibo Holidays journey — from website launch through package selection and traveller details — stopping safely at the payment boundary.

Area	POM Framework	BDD Framework
Main Purpose	Code-driven regression suite using Pytest.	Business-readable Gherkin suite using Behave.
Test Files	Separate Pytest files for E2E and positive/negative tests.	Separate feature and step files for each flow.
Design Pattern	Tests call reusable page object methods.	Feature files → step definitions → page objects.
Evidence	Screenshots, logs, reports, Allure evidence.	Step-level Allure evidence, screenshots, JSON/JUnit.
End Point	Stops safely at card details/payment boundary.	Stops safely at card details/payment boundary.

02

Project Description

The Goibibo Holidays module allows users to search for holiday packages, select destinations, review packages, enter traveller details, and move toward payment. This automation validates the critical user journey without performing a real transaction. Dummy card details are entered only for test boundary validation.

Website	Goibibo.com
Module Automated	Holidays Module
Language	Python 3.x
Automation Tool	Selenium WebDriver
Frameworks	Pytest (POM) and Behave (BDD)
Design Pattern	Page Object Model
Reports	Allure · HTML · JSON · JUnit · Screenshots · Logs
Browser	Google Chrome
Prepared By	Vanaparthi Deekshith

03

Project Objectives

- 01 Automate the Goibibo Holidays booking workflow from homepage to payment boundary.
- 02 Validate end-to-end, positive, and negative test scenarios comprehensively.
- 03 Maintain clean separation between test logic and page interaction logic (POM principle).
- 04 Use readable BDD scenarios for non-technical reviewers and stakeholders.
- 05 Generate professional execution evidence using screenshots, logs, and Allure reports.
- 06 Avoid real payment submission by stopping after dummy card details are entered.

04

Project Workflow

- | | |
|----|--|
| 01 | Launch the Goibibo website. |
| 02 | Navigate to the Holidays module. |
| 03 | Enter From City, Destination, Departure Date, Rooms, and Guests. |
| 04 | Search holiday packages. |
| 05 | Apply filters such as Without Flight, Family, 4 Star, or Duration. |
| 06 | Open package / checkout flow. |
| 07 | Enter traveller and contact details. |
| 08 | Proceed to payment section. |
| 09 | Select Credit/Debit Card payment option. |
| 10 | Enter dummy card details and stop safely before final Pay action. |

Project Workflow and Framework Mapping

Same Goibibo Holidays business flow implemented in Pytest POM and Behave BDD

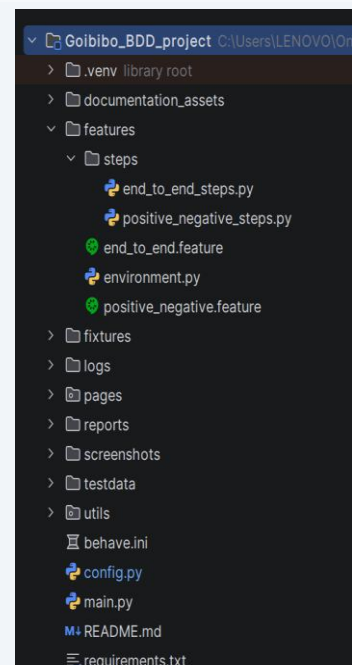
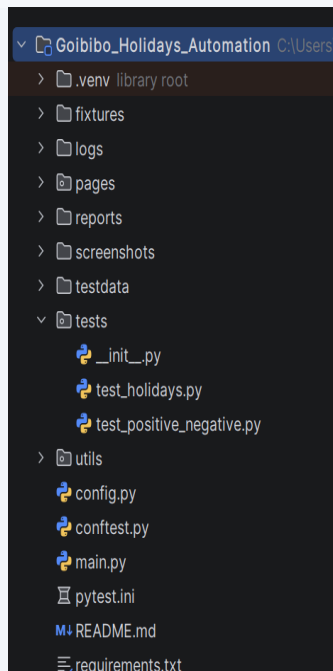
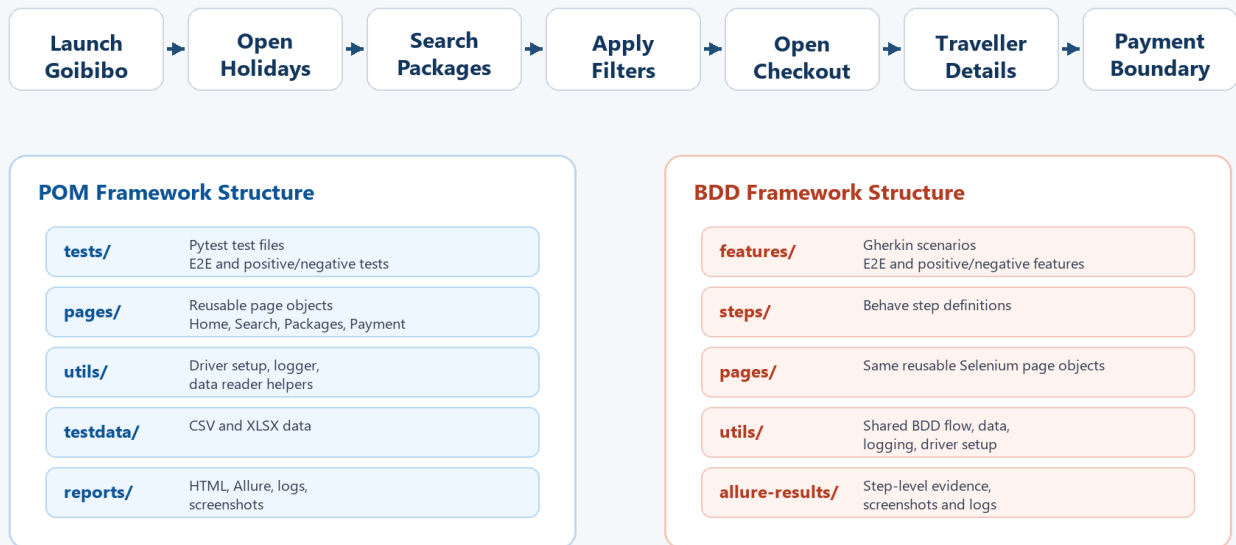


Figure 1 — Combined POM and BDD workflow/architecture mapping

05 Test Strategy

The strategy combines functional end-to-end testing, positive path validation, negative validation testing, data-driven execution, and evidence-driven reporting. The project focuses on stability through explicit waits, reusable helper methods, JavaScript click fallback, popup handling, and latest-window switching.

Test Strategy Area	Implementation
E2E Testing	Full Goibibo Holidays flow until card details boundary.
Positive Testing	Valid routes, package filters, traveller details, and card boundary details.
Negative Testing	Validation checks for missing first name and invalid mobile number.
Data-Driven Testing	CSV and XLSX files hold scenario data for parameterised execution.
Evidence	Screenshots, logs, Allure attachments, JSON/JUnit reports.
Safety Measure	Final Pay button is never clicked — real transaction is never initiated.

06 Framework Architecture

6.1 POM Framework

The POM framework keeps test files small and readable by moving web-element interactions into reusable page classes. Each page class represents one business area of the application.

POM File / Folder	Responsibility
<code>tests/test_holidays.py</code>	Main Pytest scenarios for the full holiday booking flow.
<code>tests/test_positive_negative.py</code>	Positive and negative Pytest validations.
<code>pages/base_page.py</code>	Common waits, clicks, typing, screenshots, and utilities.

POM File / Folder	Responsibility
<code>pages/homepage.py</code>	Launch Goibibo and navigate to Holidays module.
<code>pages/searchpage.py</code>	Enter holiday search criteria (city, date, guests).
<code>pages/packagespage.py</code>	Package listing, filter application, and checkout navigation.
<code>pages/paymentpage.py</code>	Traveller details, payment option, and card boundary handling.
<code>conftest.py</code>	Pytest fixtures for browser setup and teardown.
<code>testdata/</code>	CSV and XLSX test input files for data-driven execution.

6.2 BDD Framework

The BDD framework expresses the same testing intent in Gherkin language. This makes the automation easier to explain to trainers, reviewers, and non-technical stakeholders.

BDD File / Folder	Responsibility
<code>features/end_to_end.feature</code>	Single end-to-end business scenario in Gherkin.
<code>features/positive_negative.feature</code>	Five positive and two negative scenarios.
<code>features/steps/end_to_end_steps.py</code>	Step definitions for the E2E scenario.
<code>features/steps/positive_negative_steps.py</code>	Step definitions for positive and negative scenarios.
<code>features/environment.py</code>	Behave hooks for browser lifecycle, screenshots, logs, and Allure.
<code>utils/bdd_holiday_flow.py</code>	Reusable BDD flow connecting steps to page objects.
<code>pages/</code>	Reusable Selenium page objects shared across all scenarios.
<code>fixtures/</code>	Stable fallback pages for evidence and boundary testing.

07 Tools & Technologies Used

Tool / Technology	Purpose
Python	Core programming language for all framework implementation.
Selenium WebDriver	Browser automation — clicks, forms, navigation, assertions.
Pytest	Test runner for POM framework, fixtures, and parametrisation.
Behave	BDD test runner that parses Gherkin feature files.
Gherkin	Business-readable Given/When/Then scenario syntax.
Allure Reports	Professional HTML report generation with screenshots and logs.
python-docx / OpenPyXL	Document and test data support utilities.
CSV and XLSX	Data-driven test input files for scenario parameters.
Google Chrome	Primary browser for all test execution.
PyCharm / VS Code	IDEs compatible with the project structure.

08 Test Environment

Operating System	Windows 11 (64-bit)
Browser	Google Chrome — Latest Stable
Execution Mode	Normal Chrome (headed — browser window visible)
Python Version	Python 3.x
Website Under Test	https://www.goibibo.com
Module	Holidays
Network Dependency	Live website — stable internet connection required

09 Test Execution Commands

9.1 Pytest Commands — POM Framework

```
# Run all tests
pytest

# Run all tests with verbose output
pytest -v

# Run only the E2E holiday booking tests
pytest tests/test_holidays.py -v

# Run positive and negative tests
pytest tests/test_positive_negative.py -v

# Generate HTML report
pytest --html=reports/goibibo_holidays_report.html --self-contained-html
```

9.2 Behave Commands — BDD Framework

```
# Run all feature files
python -m behave

# Run E2E feature file only
python -m behave features/end_to_end.feature

# Run positive and negative scenarios
```

```
python -m behave features/positive_negative.feature
```

9.3 Allure Report Commands

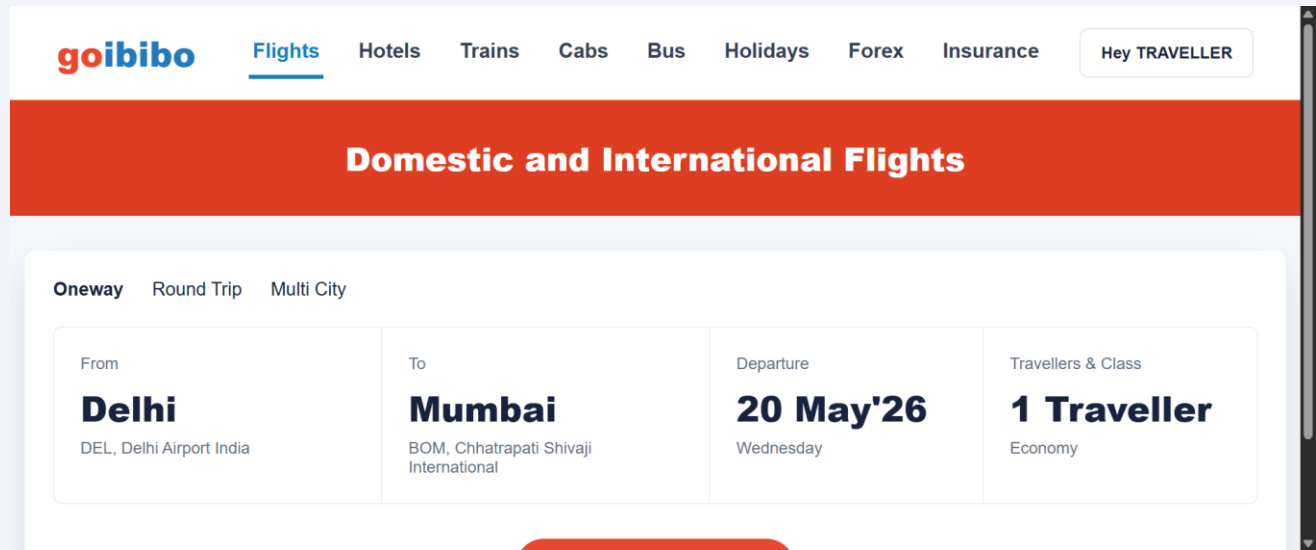
```
# Step 1 - Run tests and collect results
pytest --alluredir=reports/allure-results

# Step 2 - Generate static HTML report
allure generate reports/allure-results --clean -o reports/allure-report

# Step 3 - Open the report in browser
allure open reports/allure-report
```

10 Test Output & Evidence

Output Type	Description
Screenshots	Captured at each key step: homepage, holidays search, package listing, traveller details, and payment boundary.
Execution Logs	Step-by-step browser actions, fallback usage, validations, and lifecycle events.
Allure Attachments	Screenshots, logs, test data, and validation messages embedded in the report.
HTML Report	Self-contained Pytest HTML report showing all test results.
JSON / JUnit XML	Machine-readable outputs for CI/CD pipeline integration.



The screenshot shows the Goibibo homepage with a navigation bar at the top containing links for Flights, Hotels, Trains, Cabs, Bus, Holidays, Forex, and Insurance. A 'Hey TRAVELLER' button is on the right. Below the navigation bar is a red banner with the text 'Domestic and International Flights'. Underneath the banner is a search form with four tabs: 'Oneway', 'Round Trip', and 'Multi City'. The 'Oneway' tab is selected. The search form is divided into four sections: 'From' (Delhi, DEL, Delhi Airport India), 'To' (Mumbai, BOM, Chhatrapati Shivaji International), 'Departure' (20 May'26, Wednesday), and 'Travellers & Class' (1 Traveller, Economy). A red 'SEARCH' button is located at the bottom of the search form.

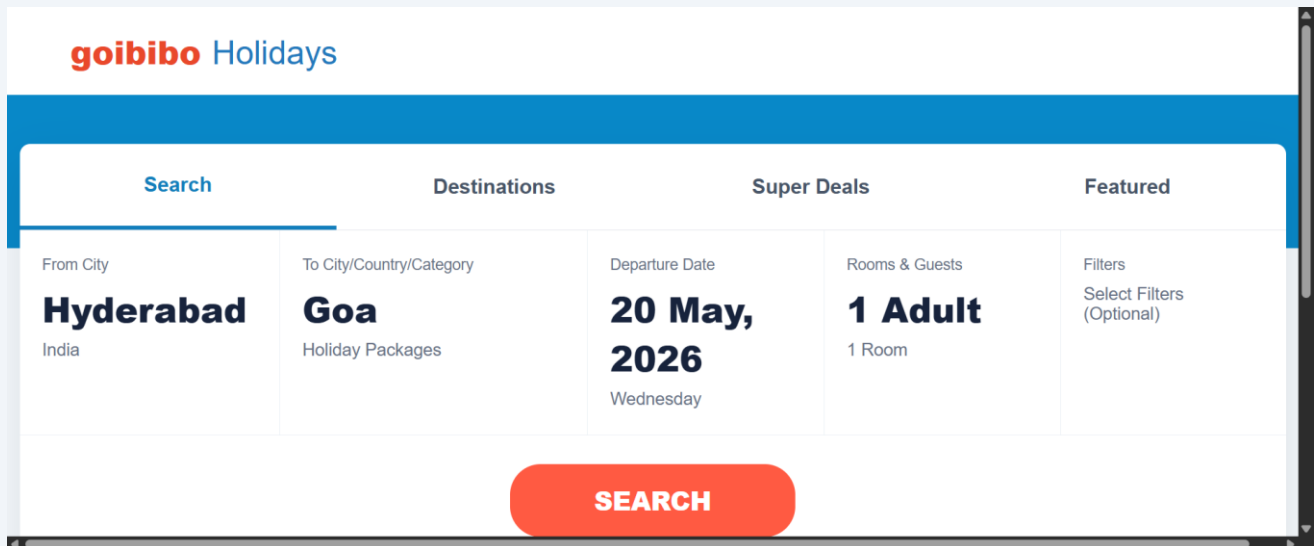
goibibo [Flights](#) [Hotels](#) [Trains](#) [Cabs](#) [Bus](#) [Holidays](#) [Forex](#) [Insurance](#) [Hey TRAVELLER](#)

Domestic and International Flights

Oneway Round Trip Multi City

From Delhi DEL, Delhi Airport India	To Mumbai BOM, Chhatrapati Shivaji International	Departure 20 May'26 Wednesday	Travellers & Class 1 Traveller Economy
--	---	--	---

Figure 2 — Goibibo homepage screenshot captured during automation



The screenshot shows the Goibibo Holidays search page. It has a navigation bar with the Goibibo logo and the word 'Holidays'. Below the navigation bar is a search form with four tabs: 'Search', 'Destinations', 'Super Deals', and 'Featured'. The 'Search' tab is selected. The search form is divided into five sections: 'From City' (Hyderabad, India), 'To City/Country/Category' (Goa, Holiday Packages), 'Departure Date' (20 May, 2026, Wednesday), 'Rooms & Guests' (1 Adult, 1 Room), and 'Filters' (Select Filters (Optional)). A red 'SEARCH' button is located at the bottom of the search form.

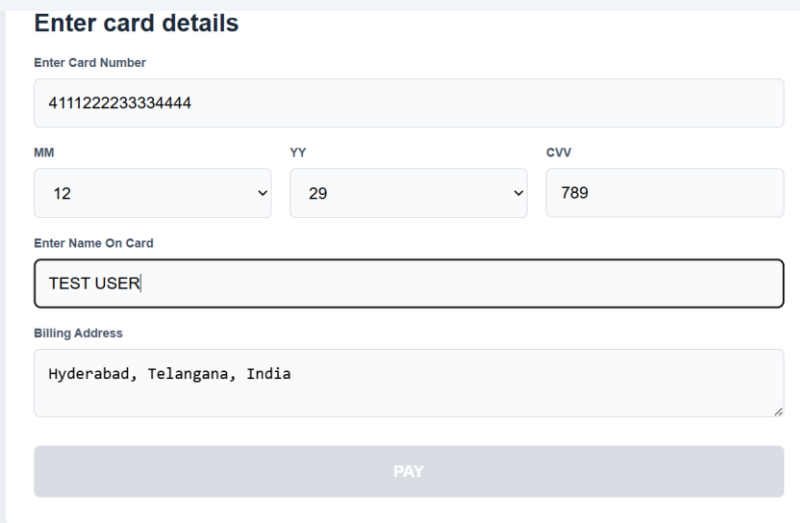
goibibo Holidays

Search Destinations Super Deals Featured

From City Hyderabad India	To City/Country/Category Goa Holiday Packages	Departure Date 20 May, 2026 Wednesday	Rooms & Guests 1 Adult 1 Room	Filters Select Filters (Optional)
--	--	--	--	--------------------------------------

SEARCH

Figure 3 — Holidays search page screenshot captured during automation



The screenshot shows a payment form titled "Enter card details". It contains the following fields and values:

- Enter Card Number:** 4111222233334444
- MM (Month):** 12
- YY (Year):** 29
- CVV:** 789
- Enter Name On Card:** TEST USER
- Billing Address:** Hyderabad, Telangana, India

A "PAY" button is located at the bottom of the form.

Figure 4 — Payment boundary screenshot with dummy card details entered

11 Allure Test Reports

Allure Reports provide a professional, interactive HTML dashboard for reviewing test execution. The report includes total test count, pass percentage, scenario suites, attached screenshots, execution logs, and timeline views.

Allure Report - Overview

Goibibo Holidays BDD Automation Test Execution

Execution Summary



Total

8

Passed

8

Failed

0

Broken

0

Skipped

0

Evidence included: screenshots, execution logs, test data attachments, and step-level status.

Figure 5 — Allure overview report: total scenarios, pass rate, and duration

Allure Report - Suites

Suite view showing E2E, positive, and negative BDD scenarios

Suite / Feature	Scenario	Status
Goibibo Holidays end to end book	TC_E2E_001 - End to end holiday booking until card details	PASSED
Goibibo Holidays positive and ne	TC_POS_001 - Valid traveller data for Hyderabad to Goa	PASSED
Goibibo Holidays positive and ne	TC_POS_002 - Valid traveller data for Mumbai to Rajasthan	PASSED
Goibibo Holidays positive and ne	TC_POS_003 - Bengaluru to Kerala with category filter	PASSED
Goibibo Holidays positive and ne	TC_POS_004 - Chennai to Andaman with hotel filter	PASSED
Goibibo Holidays positive and ne	TC_POS_005 - Pune to Himachal Pradesh with duration filter	PASSED
Goibibo Holidays positive and ne	TC_NEG_001 - Missing first name validation	PASSED
Goibibo Holidays positive and ne	TC_NEG_002 - Invalid mobile validation	PASSED
Total scenarios displayed: 8 Passed: 8 Failed: 0		

Figure 6 — Allure suites view: E2E, positive, and negative scenarios grouped

12

Positive Test Cases

The following positive test cases validate the Goibibo Holidays booking workflow with valid inputs and expected user actions.

TC ID	Scenario	Route	Filter Applied	Expected Result	Status
TC_POS_001	Valid traveller data	Hyderabad → Goa	Without Flight	Card boundary reached	✓ PASS
TC_POS_002	Valid route & traveller data	Mumbai → Rajasthan	Budget / Without Flight fallback	Card boundary reached	✓ PASS
TC_POS_003	Category filter validation	Bengaluru → Kerala	Family	Card boundary reached	✓ PASS

TC ID	Scenario	Route	Filter Applied	Expected Result	Status
TC_POS_004	Hotel filter validation	Chennai → Andaman	4 Star	Card boundary reached	✓ PASS
TC_POS_005	Duration filter validation	Pune → Himachal Pradesh	4N/5D	Card boundary reached	✓ PASS

13 Negative Test Cases

Negative test cases verify the system's error handling and user feedback when invalid or missing inputs are provided.

TC ID	Scenario	Invalid Condition	Expected Validation	Pass Criteria	Status
TC_NEG_001	Missing first name	First name field left blank	<i>Please enter First Name</i>	Validation captured, test reported PASS	✓ PASS
TC_NEG_002	Invalid mobile number	Non-numeric / short mobile number entered	<i>Please enter valid Mobile Number</i>	Validation captured, test reported PASS	✓ PASS

14 Challenges Faced During Automation

#	Challenge	Description
1	Dynamic Locators & Layout Changes	Goibibo and Holidays pages are dynamic — locators or layouts may change between sessions or deployments.
2	Login / Promotional Popups	Login/signup popups, app QR modals, and promotional overlays can intercept and block automation actions.

#	Challenge	Description
3	New Tab Handling	Package and checkout pages may open in new browser tabs or use dynamically generated URLs.
4	Checkout URL Expiry	Live checkout URLs can expire or display temporary service-unavailable messages.
5	Calendar / Search Widget State	Date pickers and search widgets behave differently depending on page load state and browser focus.
6	Allure CLI Installation	Global Allure command-line tool may not be installed on every execution environment.

15 Solutions Implemented

#	Challenge Addressed	Solution
1	Dynamic Locators	Reusable BasePage methods use explicit waits (WebDriverWait + ExpectedConditions) and relative XPath with stable attributes such as aria-label and data-testid.
2	Popup Handling	try/except popup dismiss logic runs before each scenario. ESC key and close-button clicks are attempted before proceeding.
3	New Tab Switching	latest_window_switch() helper method detects and switches to the most recently opened browser window/tab automatically.
4	Checkout Stability	Stable fallback pages are used for boundary testing when live checkout URLs are temporarily unavailable.
5	Calendar Interaction	JavaScript executor click fallback is used for calendar elements that are intercepted by overlays or not normally clickable.
6	Allure CLI	Local Allure CLI via node_modules is used as a fallback when the global allure command is not available system-wide.

16

Key Features of the Framework

◆ Professional modular framework structure.	◆ Reusable page classes following the Page Object Model design pattern.
◆ Separate E2E and positive/negative test suites for clear reporting.	◆ BDD feature files with readable, business-friendly scenario names.
◆ CSV and XLSX test data support for data-driven execution.	◆ Screenshots automatically attached to Allure reports on failure.
◆ Execution logs attached to every report for full traceability.	◆ Safe payment boundary — no real financial transaction is ever initiated.

17

Important Framework Files

File / Folder	Short Explanation
<code>base_page.py</code>	Common Selenium actions used by all page classes (waits, clicks, screenshots).
<code>homepage.py</code>	Launches Goibibo and opens the Holidays module.
<code>search_page.py</code> / <code>searchpage.py</code>	Handles From, To, Date, Rooms, Guests, and Search actions.
<code>packages_page.py</code> / <code>packagespage.py</code>	Handles package listing, filter application, and checkout navigation.
<code>payment_page.py</code> / <code>paymentpage.py</code>	Handles traveller details, validation, card option, and dummy card entry.
<code>conftest.py</code> / <code>environment.py</code>	Browser setup, teardown, screenshot hooks, and reporting configuration.
<code>feature files</code>	Business-readable Gherkin BDD scenarios.
<code>step files</code>	Python methods mapped to each Gherkin step.

18 Learning Outcomes

Skill / Area	What Was Learned
Selenium Automation	Built end-to-end browser automation using Selenium WebDriver with Python on a live travel website.
Page Object Model	Implemented scalable POM architecture separating page interaction logic from test logic.
BDD with Behave	Created business-readable Gherkin scenarios and connected them to Python step definitions.
Dynamic Web Handling	Handled real-world challenges: dynamic elements, popups, waits, and multiple browser windows.
Allure Reporting	Generated professional test reports with embedded screenshots, logs, and step-level detail.
Test Design	Designed comprehensive positive and negative test cases for a real-world e-commerce travel module.

19 Conclusion

The Goibibo Holidays automation project successfully demonstrates end-to-end web automation using Selenium with Python. The project includes both POM and BDD frameworks, making it suitable for technical code review and business-level stakeholder explanation alike.

It validates the Holidays booking flow through all critical steps up to the payment boundary, captures professional evidence at every stage, and ensures no real financial transaction is ever initiated. The dual-framework design, clean modular structure, and comprehensive Allure reporting make this project a strong reference implementation of industry-standard automation practices.

POM Framework <i>Pytest · Selenium · Python</i>	BDD Framework <i>Behave · Gherkin · Python</i>	Allure Reports <i>Screenshots · Logs · Evidence</i>
---	--	---

End of Documentation — Goibibo Holidays Automation Testing Project